Preregistered Paired Evaluation of a Deterministic Verifier Pipeline for LLM-Generated Network Configuration Changes — Non-informative Result under Shared-Generation Semantics

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We report a fully preregistered paired experiment (cand-2026-01-prereg-v1) that tested whether a deterministic verifier pipeline (generate → deterministic_netops_validator_v5 → optional repair) reduces the rate of verifier-detected unsafe intent→configuration proposals produced by a hosted LLM on synthetic NetOps tasks. Design: N=200 preregistered unique intent→configuration tasks in a paired design comparing (a) baseline LLM-only and (b) guarded pipeline (gpt-5-mini + deterministic_netops_validator_v5). Execution used shared_initial_candidate generation semantics (one LLM call per task) producing model_calls_used = 200 (preregistered independent per-arm generation would have used up to 400). Observed (adversarial cluster): baseline SAFE = 200/200; guarded SAFE = 200/200 (paired contingency n11 = 200, n10 = 0, n01 = 0). Estimated paired SAFE-rate difference = 0.0; exact McNemar two-sided p = 1.0; paired bootstrap 95% CI = [0.0, 0.0] (resamples = 10000, seed = 1729). Because identical per-pair generations were produced and the observed baseline unsafe rate was 0%, the preregistered causal hypothesis (that the deterministic verifier reduces unsafe proposals) was not testable in this execution. We publish the complete preregistered run and artifacts, provide an artifact-grounded diagnosis of testability failure, and give concrete, archive-supported recommendations (independent per-arm generation budgeting, adversarial injection calibration, and exploratory ROC reserves) for informative repeats. All execution and analysis artifacts cited here are archived in the supplied bundle.**

I. Introduction

Automating human-intent → device-configuration translation with large language models (LLMs) is an active NetOps research thrust because it can reduce human effort and speed maintenance tasks. Yet incorrectly specified or misordered changes can cause outages, policy violations, or security exposures. A commonly proposed mitigation is tool-augmentation: couple an LLM generator to deterministic validators, sandboxed simulators, or constrained executors in a generate–validate–repair pipeline so that proposed changes are checked before execution.

We preregistered and executed a confirmatory paired experiment (cand-2026-01-prereg-v1) to quantify whether deterministic verifier augmentation causally reduces the frequency of verifier-detected unsafe proposals from a hosted LLM under both benign and adversarial prompt clusters. The preregistered estimand was the paired SAFE-rate difference (guarded minus baseline), tested by an exact McNemar two-sided test with a paired bootstrap CI (10000 resamples, seed = 1729). The design sampled N=200 tasks using a deterministic generator and a paired comparison across two pipelines. Two generation semantics were permitted in the preregistration: independent per-arm generation (one LLM call per arm per task) or shared_initial_candidate (one shared LLM call per task scored by both arms). The executed run used shared_initial_candidate to conserve model-call budget.

Key outcome: under the executed shared semantics the sampled LLM outputs were scored SAFE by the deterministic validator in every confirmatory episode (baseline SAFE = 200/200; guarded SAFE = 200/200). This concordant ceiling outcome (n11 = 200; n10 = n01 = n00 = 0) yields an estimate of zero paired difference and an exact McNemar p = 1.0. Because the observed baseline unsafe rate was 0% and the same candidate was used in both arms, the preregistered causal hypothesis could not be meaningfully evaluated. The manuscript therefore focuses on three contributions grounded in archived artifacts: (1) a complete, deterministic preregistered run published as reproducible artifacts; (2) a precise, artifact-grounded diagnosis of why this execution was non-informative for the confirmatory estimand; and (3) concrete, artifact-supported design recommendations that would enable informative repeats under the same preregistration framework.

II. Related Work

Prior surveys and prototype reports document rapid uptake of LLMs in NetOps workflows (intent translation, configuration synthesis, diagnosis and limited remediation), and they emphasize both promise and safety concerns when models produce executable changes [https://openalex.org/W7161354925; https://openalex.org/W7170714602; https://doi.org/10.36227/techrxiv.177004277.71795055/v1]. Those surveys consistently note that measured correctness gains from LLMs are often task-dependent and evaluated on curated case sets rather than on replayable workflow-centred evaluations.

A recurring architectural pattern in the literature is tool-augmentation: pairing an LLM generator with determin-

istic tools (validators, simulators, or constrained executors) in generate–validate–refine or divide–verify–refine pipelines. Experimental reports and method proposals show that these pipelines can reduce explicit constraint violations on curated tasks and improve adherence to complex instruction structure when the external verifier/simulator faithfully encodes domain constraints [https://openalex.org/W4412888330; https://openalex.org/W7161354925]. However, the reported improvements depend strongly on the fidelity of the verifier and on the evaluation semantics (how candidate generations are sampled, whether repairs are applied, and whether validation executes in a sandboxed replay), so cross-study comparisons require careful alignment of these elements.

Work on guardrails, schema-based tool interfaces, and secure agent orchestration stresses practical trade-offs: stricter schemas or deterministic checks can lower acceptance of unsafe proposals but may increase false refusals or operator burden if the validator is incomplete or overly conservative [https://doi.org/10.36227/techrxiv.177006109.97957916/v1; https://doi.org/10.36227/techrxiv.177004277.71795055/v1]. Red-teaming and adversarial evaluation studies from adjacent domains further demonstrate that guardrail designs must be stress-tested with adversarial templates and that evaluation protocols should report both failure-to-detect (false negatives) and false-refusal (false positive) rates to capture operational trade-offs; several position and empirical papers advocate ROC-style diagnostics for verifier behaviour where feasible [https://doi.org/10.2139/ssrn.7132138; https://openalex.org/W7161354925].

Evaluation methodology literature for agentic NetOps emphasizes workflow-centred, replayable, and sandboxed experiments: tasks should include explicit initial and expected final states, deterministic topology generators, and archived validator traces so that runs are reproducible and auditors can replay failures in a sandboxed environment. Several recent proposals and benchmarks articulate these desiderata and argue that static QA metrics are insufficient when the downstream risk involves stateful multi-device changes or policy composition across devices [https://doi.org/10.1109/ojcoms.2026.3680457; https://openalex.org/W7161354925].

Our study aligns with this body of work by (a) using a deterministic task generator and sandboxed deterministic validation, (b) preregistering a paired estimand and detailed exclusion/missingness rules, and (c) archiving full validator traces and execution manifests to enable replay. It also addresses an empirical evaluation gap identified in the literature: although tool-augmentation is widely advocated and shown to help in many reports, there are relatively few reproducible, preregistered adversarial NetOps experiments that quantify the paired causal effect of adding a deterministic verifier under controlled generation semantics. The papers cited above both motivate such paired designs and underscore the sensitivity of conclusions to evaluation semantics (sampling, repair policies, and generation budgeting), which is the methodological locus of our artifact-grounded diagnosis in this manuscript [https://openalex.org/W4412888330; https://openalex.org/W7161354925; https://doi.org/10.36227/techrxiv.177004277.71795055/v1].

III. Methodology

Preregistration and estimand. The experiment was preregistered as cand-2026-01-prereg-v1. Primary estimand: paired_success_rate_difference_guarded_minus_baseline (guarded - baseline) on binary SAFE indicators obtained from a deterministic sandbox validator. Confirmatory test: exact two-sided McNemar implemented by paired_binary_analysis_v1; confidence intervals: paired bootstrap percentile (10000 resamples, seed = 1729). The preregistration specified a paired binary design with N=200 tasks, planned paired episodes = 400, and a power plan that targeted detecting an approximate 7 percentage-point reduction in unsafe proposals with 80% power assuming a baseline unsafe rate ≈10–15%.

Task generator, scope, and seeds. A deterministic generator (deterministic_netops_task_generator_v5) produced 200 synthetic intent→configuration tasks constrained to single-AS multi-device topologies and limited configuration fields (admin, mtu, vlan). Generator seeds were deterministic (per-task generation_seed 1..200; the RNG seed for the generator family was preregistered) and each task manifest included initial_state, expected_final_state, explicit per-task workflow policies (required_sequence), and difficulty metadata. Tasks were clustered by benign vs adversarial prompt transforms using a preregistered obfuscation template set to increase adversarial difficulty; the adversarial cluster was included in confirmatory items per the preregistration.

Pipelines compared. Baseline: accept the LLM candidate (no runtime deterministic enforcement) and score post hoc with deterministic_netops_validator_v5. Guarded: validate the same LLM candidate in a deterministic sandbox using deterministic_netops_validator_v5; the validator emitted structured traces (validation_before/after, violation lists) and optionally could invoke one automatic repair call per the preregistration. For confirmatory episodes no repairs were applied (repair fields are null in scoring JSONs) so the guarded result reflects pure validation rather than repair-induced change.

Generation semantics and model budgeting. The preregistered plan allowed two generation semantics: independent per-arm generation (two model calls per task) or shared_initial_candidate (one model call per task whose candidate is scored by both arms). The executed run used shared_initial_candidate to conserve hosted model budget. Planned maximum_model_calls for independent generation = 400; executed model_calls_used = 200. This single shared candidate per task meant that, absent repair, the two arms would always score the same candidate and could not produce discordant paired outcomes arising from arm-level stochasticity.

Scoring and analysis pipeline. Deterministic_netops_validator_v5 applied full_intent_constraint_validation to each candidate and emitted a deterministic valid flag and detailed violation

lists. Confirmatory binary outcomes were derived from validator.valid (1 = SAFE, 0 = UNSAFE). Analysis used paired_binary_analysis_v1 to compute the 2×2 contingency counts (n11,n10,n01,n00), exact McNemar two-sided p-value, and paired bootstrap CI (10000 resamples, seed = 1729). Missingness and retry rules from the preregistration were applied: one automatic retry per failed model call; contaminated or nondeterministic tasks are excluded and replaced. Deterministic reconciliation checks and provider audit were recorded as analysis artifacts and used to certify that no technical failures, exclusions, or nondeterminism occurred in the confirmatory run.

IV. Results

Execution and provider accounting (artifact pointers). The execution manifest records completed_episode_count = 400 (200 paired episodes), failed_episode_count = 0, and model_calls_used = 200 (see execution/execution_manifest.json and execution/model_configuration.json for declared model and generation semantics). Deterministic_reconciliation.json and execution/execution_manifest.json document that hosted_model_call_event_count = 200, cache_miss_count = 200, and that all deterministic consistency checks passed; no episodes were excluded or flagged (analysis/contamination_summary.csv reports zero flagged episodes) and analysis/missingness_summary.csv reports complete_pairs = 200.

Primary 2×2 paired outcomes and exact inference. The confirmatory contingency table derived by paired_binary_analysis_v1 is: n11 = 200 (SAFE in both arms), n10 = 0 (SAFE guarded, UNSAFE baseline), n01 = 0 (SAFE baseline, UNSAFE guarded), n00 = 0 (UNSAFE both). These counts are archived in analysis/paired_contingency_table.csv. From these counts the point estimate for the paired SAFE-rate difference (guarded - baseline) equals 0.0. The preregistered exact McNemar two-sided test yields p = 1.0 under the observed contingency. The paired bootstrap percentile 95% confidence interval computed by the analysis executor (10000 resamples, seed = 1729) is degenerate at [0.0, 0.0]; the analysis log and bootstrap parameters are archived in analysis/analysis_log.jsonl and analysis/paired_difference_figure.svg.

Representative per-task diagnostics (scoring JSON evidence). Representative scoring artifacts (execution/scoring/task-000001-baseline.json and task-000001-guarded.json) demonstrate that the shared_initial_candidate content was identical across arms and that the validator trace fields validation_before and validation_after both report valid = true and violation_count = 0. Repair provider traces are null in all confirmatory scoring JSONs (repair_provider_trace_path = null), confirming that no automatic repair calls altered any candidate during confirmatory episodes. Representative shared_initial response files (execution/responses/task-000001-shared-initial.txt, task-000002-shared-initial.txt, task-000003-shared-initial.txt) contain the canonical configurations used for scoring; these files match the normalized_response fields in the scoring JSONs.

Detailed validation metadata. Each scoring JSON embeds validator metadata: scorer_id = deterministic_netops_validator_v5, scorer_version = 5.0, scoring_rule = full_intent_constraint_validation, and scoring_status = COMPLETED. Validator traces include parsed_command_count, required_command_count, observed_touched_field_count, and difficulty annotations (e.g., level = medium/hard/very_hard) for each task; these per-task difficulty tags are archived in execution/task_manifest.jsonl and appear in the representative task entries. The validator reported zero violation_codes and an empty violations array in every confirmatory episode's validation_after block, consistent with the binary SAFE labels used for the paired analysis.

Why the confirmatory test was uninformative (artifact-grounded diagnosis). Three concrete archived facts explain the degenerate contingency: (1) the run executed shared_initial_candidate semantics (independent_condition_generation = false in execution/model_configuration.json) so both arms scored the same single LLM output per task; (2) the sampled gpt-5-mini outputs were labeled SAFE by deterministic_netops_validator_v5 for every confirmatory task (baseline SAFE = 200/200 as recorded in analysis/condition_summary.csv); and (3) no repair calls were applied in confirmatory episodes (repair_provider_trace fields are null), so the guarded arm could not produce arm-specific candidate changes. Together these facts produce perfect concordance (n11 = 200) and therefore zero observed discordant pairs, which makes McNemar inference vacuous (p = 1.0) and yields a degenerate bootstrap CI.

Power assumptions vs observed data. The preregistered power calculation assumed a nonzero baseline unsafe rate (~10–15%) and discordant proportions approximated by p10 = 0.07 and p01 = 0.01 (preregistration text), yielding a required sample size $\approx$ 174 for 80% power to detect a ~7 percentage-point paired reduction. The executed shared semantics reduced model calls (200 used vs planned 400 for independent per-arm draws) and removed arm stochasticity; consequently the empirical baseline unsafe rate observed here (0%) contradicts the power assumptions and collapses inferential sensitivity. This is an execution-semantic and budgeting outcome rather than evidence about verifier effectiveness.

Exploratory reserves and uncomputed diagnostics. The preregistration reserved up to 50 exploratory tasks to estimate verifier ROC and refusal trade-offs; that reserve was not consumed before confirmatory execution and therefore no empirical ROC or refusal-rate estimates were produced in the archived confirmatory artifacts. Analysis/contamination_summary.csv and analysis/missingness_summary.csv document that no exploratory ROC artifacts were appended to the confirmatory results. The absence of ROC or refusal data is an archival fact in the

supplied bundle and limits secondary claims about verifier true-positive/false-negative behavior.

Sensitivity and reproducibility checks. Deterministic_reconciliation.json shows consistent episode accounting (complete_pair_count = 200; n_11 = 200; provider_call_audit.hosted_model_call_event_count = 200). Analysis artifacts (paired_contingency_table.csv, condition_summary.csv, analysis_log.jsonl) are self-consistent and sufficient to reproduce the numeric inference reported here. All raw scoring JSONs, shared_initial responses, and the model_configuration/execution_manifest artifacts are archived in the bundle and permit deterministic replay of the exact shared-generation execution semantics that produced the degenerate contingency.

V. Discussion

We expand the original diagnosis with technical detail and artifact-grounded interpretation to help others design informative repeats that avoid the ceiling/testability failure observed here.

Mechanism of the ceiling failure. The confirmatory estimand (paired_success_rate_difference_guarded_minus_baseline) depends on arm-level discordance: McNemar-style inference requires nonzero counts in the off-diagonal cells (n10 and/or n01). In our execution the pipeline semantics used shared_initial_candidate (one LLM output per task that both arms scored). Because no repairs were applied in confirmatory episodes and the validator gave identical deterministic verdicts for the shared candidate, every paired outcome fell into n11 (SAFE in both arms). This deterministic concordance eliminates the very data structure McNemar inference needs and yields a vacuous test (p = 1.0) and a degenerate bootstrap CI [0.0,0.0]. The archived artifacts that directly demonstrate this mechanism include execution/model_configuration.json (declared independent_condition_generation = false) and analysis/paired_contingency_table.csv (n11 = 200, n10 = n01 = n00 = 0).

Budgeting and statistical power implications. The preregistration explicitly considered two generation semantics and a target power calculation predicated on a nonzero baseline unsafe rate (≈10–15%) and expected discordant proportions (e.g., preregistered p10=0.07, p01=0.01 leading to N≈174). Independent per-arm generation (two independent draws per task) is necessary to produce arm-level stochastic contrasts when the intervention (validator presence) is applied post-generation and repairs are not used to change candidates. Independent per-arm draws at our planned sample size would increase model calls from the executed 200 to the preregistered 400. The execution artifact model_calls_used = 200 versus the preregistered maximum_model_calls = 400 documents the exact budgetary shortfall that removed per-arm stochasticity and thus collapsed statistical sensitivity.

Role of repairs and alternative discordance mechanisms. Two possible, protocol-compatible mechanisms could have produced discordant pairs without independent per-arm draws: (1) enable guarded-arm repairs that modify a candidate flagged UNSAFE into a SAFE configuration and (2) use a verifier that can nondeterministically refuse while baseline accepts (or vice versa). In our archived confirmatory episodes the repair_provider_trace fields are null and no repairs were applied; consequently repairs were not a source of discordance. If repairs are allowed in the guarded arm, a different estimand (e.g., paired probability that guarded produces SAFE after repair while baseline remains UNSAFE) should be preregistered and analyzed with contingency cells defined appropriately. The scoring JSONs and repair fields in execution/scoring/*.json make the absence of repair activity explicit for this run.

Task difficulty and validator coverage diagnostics. Representative task manifests (archived in execution/task_manifest.jsonl) include per-task difficulty metadata. In the six representative tasks included in the manuscript bundle the preregistered difficulty labels are: medium (1), hard (2), very_hard (3). These mixed difficulty labels indicate the generator produced nontrivial task patterns (e.g., multi-step shutdown/configure/restore or make-before-break switchover) even though the sampled gpt-5-mini outputs satisfied the validator on every confirmatory item. This suggests two nonexclusive interpretations consistent with archived evidence: (a) the LLM generated valid step sequences for these task templates reliably under the provided prompts and model setting; or (b) the deterministic_netops_validator_v5 abstractions and the synthetic task family together created a validation surface for which the model's current capabilities were sufficient. The validator traces (validation_before/after) in execution/scoring/*.json show required_command_count and parsed_command_count matching expected sequences for representative tasks, supporting the artifact-grounded observation that produced candidates met the formalized intent constraints for the sampled items. However, the validator's completeness relative to real-world NetOps failure modes remains a separate threat to external validity (see Limitations).

Practical design prescriptions supported by artifacts. From the recorded preregistration assumptions and execution manifest we make three concrete, archive-supported prescriptions for informative repeats: - Prefer independent per-arm generation when the estimand tests the effect of adding a deterministic verifier to generation without repairs. For N=200 paired tasks this requires roughly doubling model calls from 200 to ~400 (compare execution/model_configuration.json and execution/execution_manifest.json). This change restores per-arm stochasticity and permits nonzero off-diagonal counts to appear when the validator interacts with model variability. - Use the preregistered exploratory reserve (up to 50 tasks) to calibrate adversarial transforms until a nonzero baseline unsafe rate consistent with the power plan is observed. Archive the exploratory runs and ROC estimates before committing to the confirmatory run; the preregistration and analysis artifacts show that the exploratory reserve was available but unused here. - If repairs are permissible in the guarded arm, preregister a confirmatory estimand that explicitly ac-

counts for repair conversions (UNSAFE→SAFE) and instrument repair_provider_trace fields for counting conversions and side effects; ensure repairs themselves are deterministically replayable and archived (repair traces were null in this execution—see execution/scoring/*.json). These prescriptions are not speculative: they are directly motivated by and supported in the archived preregistration, execution manifest, and analysis outputs.

Statistical interpretability and reporting guidance. When a ceiling or floor outcome appears (all observations in one cell), standard hypothesis tests produce noninformative outputs (exact p = 1.0, degenerate CIs). We recommend that authors (a) report the contingency table and raw counts prominently (analysis/paired_contingency_table.csv), (b) predeclare and run sensitivity analyses that treat technical failures as safe or unsafe per the preregistration, and (c) document any shared-generation semantics and model-call budget choices in the Methods (execution/model_configuration.json). Doing so makes it clear when a null statistical result owes to experimental semantics rather than substantive equivalence of pipelines.

Operational implications and limits of inference. The present run is evidence that, for the sampled synthetic tasks and generation setting, gpt-5-mini produced validator-compliant configurations under the specified prompts; it is not evidence that deterministic verification is unnecessary in general. The degeneracy of the contingency table prevents statements about relative safety or missed unsafe proposals. Practitioners seeking to evaluate verifier utility in operational deployments should (i) decide whether the estimator of interest is effect of validator presence on independent generation or effect of repair conversion, (ii) allocate generation budget accordingly, and (iii) tune adversarial strength via held-out exploratory trials. All these recommendations are directly traceable to the archived preregistration and execution artifacts.

Reproducibility and artifact pointers. The diagnosis above is fully reproducible from the archived bundle: the generation semantics and model_call accounting appear in execution/model_configuration.json and execution/execution_manifest.json; the paired contingency and condition summaries are in analysis/paired_contingency_table.csv and analysis/condition_summary.csv; representative scoring traces are in execution/scoring/*.json. Researchers planning informative repeats should consult these artifacts first to avoid repeating the same semantic mismatch between estimand and execution.

VI. Limitations

- Synthetic tasks and scope: tasks were deterministic synthetic topologies produced by deterministic_netops_task_generator_v5 and limited to single-AS, multi-device setups; results may not generalize to production, multi-AS, or vendor-specific features.
- Generation semantics: the executed run used shared_initial_candidate (independent_condition_generation = false), producing identical per-pair generations and creating a ceiling effect when shared candidates were valid.
- Adversarial strength: the preregistered adversarial templates used in this run did not elicit unsafe outputs from gpt-5-mini on the sampled tasks; stronger adversarial cases or explicit injected unsafe examples are required to measure verifier effect.
- Single-model scope: only the declared model (gpt-5-mini) was evaluated; no multi-model generality is claimed.
- Verifier completeness: deterministic_netops_validator_v5 ran deterministically in synthetic sandboxed states; external validation of validator completeness against all real-world NetOps failure modes is not provided.
- Operational external validity: no production deployment, operator-in-the-loop workload measurement, nor multi-vendor field trials were performed; operator-cost trade-offs remain unquantified at scale.

VII. Conclusion

We executed a fully archived preregistered paired experiment comparing gpt-5-mini baseline generation and a deterministic verifier-augmented pipeline on 200 synthetic NetOps tasks. Under the executed shared_initial_candidate semantics and for the declared model, both arms produced zero verifier-detected unsafe proposals (paired difference = 0.0; exact McNemar p = 1.0; 95% CI = [0.0, 0.0]). Because identical per-pair generations were used and because the observed baseline unsafe rate was 0%, the preregistered causal hypothesis was not testable in this run. The scientific contribution is therefore methodological and reproducibility-centred: (1) a complete deterministic preregistered run with archived artifacts that permit exact replay; (2) an artifact-grounded diagnosis of why this execution was non-informative for verifier efficacy; and (3) concrete, archive-supported recommendations (independent per-arm generation budgeting, adversarial calibration, and exploratory ROC estimation) for informative repeats. The archived execution and analysis artifacts cited throughout (execution/* and analysis/*) permit deterministic replication of the diagnosis and the run outputs and should be consulted prior to attempting repeats or production extrapolation.

Disclosure Statement

Disclosure Statement (CNSM Agentic AI Researcher track): Declared LLM: gpt-5-mini (execution recorded in execution/model_configuration.json). Orchestration/framework: hosted_netops_gvr_v1 adapter and deterministic experiment harness (execution/execution_manifest.json). Exact initial master prompt: archived at provenance/master_prompt.txt with immutable SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15 Topic constraints and autonomy boundary: the human Research Director selected the broad topic family (Generative AI and LLMs for NetOps) prior to the autonomy lock; after the lock the autonomous researcher performed

literature review, preregistration (cand-2026-01-prereg-v1), experiment design, code generation, execution, analysis, manuscript drafting, autonomous peer review, and revisions. Preregistration and execution: the preregistration was frozen before execution; key preregistered elements (estimand, paired design N=200, task generator, allowed generation semantics, scoring rules, and stopping rules) are recorded in cand-2026-01-prereg-v1 and archived in the manuscript bundle. Generation semantics and model-call accounting (executed): independent_condition_generation = false (shared_initial_candidate semantics) producing a single initial LLM candidate per task shared across arms; model_calls_used = 200 while the preregistered maximum_model_calls allocated for independent per-arm generation = 400 (execution/execution_manifest.json and execution/model_configuration.json). Validator and scoring: deterministic_netops_validator_v5 performed deterministic sandboxed validation and encoded outcomes as binary labels (1 = SAFE, 0 = UNSAFE) under scoring_rule = full_intent_constraint_validation; archived scoring JSONs (execution/scoring/*.json) contain validator traces showing validation_before/after and violation lists. Analysis executor: paired_binary_analysis_v1 computed the paired contingency, exact McNemar two-sided test, and paired bootstrap CI (resamples = 10000, seed = 1729); analysis artifacts include analysis/paired_contingency_table.csv and analysis/condition_summary.csv. Deviations and restarts: no post-preregistration design changes were executed; the observed model call count reflects the executed shared_initial_candidate semantics. Reproducibility pointers (concise): execution/raw_results.jsonl, execution/task_manifest.jsonl, execution/model_configuration.json, execution/execution_manifest.json, analysis/paired_contingency_table.csv, analysis/condition_summary.csv, and analysis/paired_difference_figure.svg are archived in the supplied bundle and permit deterministic replays. Human editing: no manual human edits to technical content, analyses, or manuscript text occurred after the autonomy lock. Pre-lock development: infrastructure rehearsals occurred prior to the lock but were excluded from final empirical evidence unless reproduced under the post-lock execution.

REFERENCES

[1] https://arxiv.org/abs/2605.12729
[2] https://doi.org/10.2139/ssrn.7132138
[3] https://doi.org/10.36227/techrxiv.177004277.71795055/v1
[4] https://doi.org/10.36227/techrxiv.177006109.97957916/v1
[5] https://doi.org/10.1109/ojcoms.2026.3680457
[6] Xianren Zhang, Xianfeng Tang, Hui Liu, Zongyu Wu, Qi He, Dongwon Lee, Suhang Wang, "Divide-Verify-Refine: Can LLMs Self-align with Complex Instructions?", 2025, 10.18653/v1/2025.findings-acl.709
[7] Muhammad Bilal, Jon Crowcroft, Ruizhi Wang, Xiaolong Xu, Schahram Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety", 2026